

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	Ch.saisri	Reg. No.:	192472367
Section / Batch:	Cse-ai	Date:	

Code of Conduct

I **CH.SAISRI** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A conversational AI system must identify grammatical relationships in user queries containing long-distance dependencies. The developers are comparing constituency parsing and dependency parsing.

Analyze how the two representations differ in capturing hierarchical and word-to-word relationships, and justify which approach is more suitable for extracting meaningful relationships from conversational text.

2. An NLP system processes partially typed search queries such as “best hotels near...” and “book a flight from Chennai to...”. The system currently uses conventional top-down parsing, while the developers are considering Earley parsing.

Analyze how the two approaches behave when input is incomplete or ambiguous, and justify which parsing strategy is more appropriate for interactive NLP applications.

3. A grammar-checking system must distinguish between multiple interpretations of structurally ambiguous sentences. The developers are comparing CFG, PCFG, and neural constituency parsers.

Analyze how each approach represents and resolves syntactic ambiguity, and justify which approach would provide the best balance between interpretability and practical accuracy.

4. A multilingual NLP system must process sentences in which grammatical information depends on number, gender, tense, and person. The developers are considering feature structures and traditional context-free grammar rules.

Analyze how feature structures extend basic grammar representations and justify their usefulness for enforcing grammatical constraints in multilingual systems.

5. A voice assistant must interpret commands containing verbs with different argument requirements, such as “give the book to John,” “sleep,” and “put the file on the desk.” The developers are considering subcategorization frames.

Analyze how subcategorization information helps determine valid sentence structures and justify its importance in semantic and syntactic analysis.

Q1: CONSTITUENCY VS DEPENDENCY PARSING

```
import nltk
```

```
nltk.download("punkt", quiet=True)
```

```
nltk.download("punkt_tab", quiet=True)
```

```
nltk.download("averaged_perceptron_tagger", quiet=True)
```

```
nltk.download("averaged_perceptron_tagger_eng", quiet=True)
```

```
from nltk import word_tokenize, pos_tag
```

```
print("=" * 60)
```

```
print("CONSTITUENCY AND DEPENDENCY ANALYSIS")
```

```
print("=" * 60)
```

```
sentence = "The student booked a flight from Chennai to Delhi."
```

```
print("\nSentence:")
```

```
print(sentence)
```

```
# Tokenization
```

```
words = word_tokenize(sentence)
```

```
print("\nTokens:")
```

```
print(words)
```

```
# POS tagging
```

```
tags = pos_tag(words)
```

```
print("\nPOS Tags:")
```

```
for word, tag in tags:
```

```
    print(word, "->", tag)
```

```
# Constituency concept
```

```
print("\n" + "=" * 60)
```

```
print("CONSTITUENCY REPRESENTATION")
```

```
print("=" * 60)
```

```
print("""
```

Sentence

└─ Noun Phrase (NP)

| └─ The student

└─ Verb Phrase (VP)

└─ booked

└─ Noun Phrase

| └─ a flight

└─ Prepositional Phrase

└─ from Chennai to Delhi

```
""")
```

Dependency concept

```
print("=" * 60)
```

```
print("DEPENDENCY REPRESENTATION")
```

```
print("=" * 60)
```

```
print("""
```

booked

└─ student -> Subject

└─ flight -> Object

└─ Chennai -> Source

└─ Delhi -> Destination

```
""")
```

```
print("Result:")

print("Constituency parsing shows hierarchical structure.")

print("Dependency parsing shows word-to-word relationships.")

print("Dependency parsing is useful for extracting relationships.")
```

OUPUT:

CONSTITUENCY AND DEPENDENCY ANALYSIS

Sentence:

The student booked a flight from Chennai to Delhi.

Tokens:

['The', 'student', 'booked', 'a', 'flight', 'from', 'Chennai', 'to', 'Delhi', '.']

POS Tags:

The -> DT
student -> NN
booked -> VBD
a -> DT
flight -> NN
from -> IN
Chennai -> NNP
to -> TO
Delhi -> NNP
. -> .

CONSTITUENCY REPRESENTATION

Sentence

```
├─ Noun Phrase (NP)
│   └─ The student
├─ Verb Phrase (VP)
│   ├── booked
│   ├── Noun Phrase
│   │   └─ a flight
│   └─ Prepositional Phrase
│       └─ from Chennai to Delhi
```

DEPENDENCY REPRESENTATION

booked

```
├─ student      -> Subject
├─ flight       -> Object
├─ Chennai      -> Source
└─ Delhi        -> Destination
```

Result:

Constituency parsing shows hierarchical structure.
Dependency parsing shows word-to-word relationships.
Dependency parsing is useful for extracting relationships.

Q2: TOP-DOWN VS EARLEY PARSING

```
import nltk

from nltk import CFG

from nltk.parse import EarleyChartParser

nltk.download("punkt", quiet=True)

nltk.download("punkt_tab", quiet=True)

print("=" * 60)

print("TOP-DOWN AND EARLEY PARSING")

print("=" * 60)

sentence = "best hotels near"

print("\nIncomplete Query:")

print(sentence)

words = nltk.word_tokenize(sentence.lower())

print("\nTokens:")

print(words)

# Simple grammar

grammar = CFG.fromstring("""

S -> NP

NP -> ADJ NOUN PP
```

PP -> PREP

ADJ -> 'best'

NOUN -> 'hotels'

PREP -> 'near'

""")

Earley Parser

parser = EarleyChartParser(grammar)

print("\n" + "=" * 60)

print("EARLEY PARSER")

print("=" * 60)

try:

trees = list(parser.parse(words))

print("Number of parses:", len(trees))

if trees:

print("\nPartial/Available Parse:")

print(trees[0])

else:

print("No complete parse.")

except ValueError as e:

print("Input is incomplete.")

```
print("Earley parsing can maintain partial parsing states.")
```

```
# Comparison
```

```
print("\n" + "=" * 60)
```

```
print("COMPARISON")
```

```
print("=" * 60)
```

```
print("""
```

Top-Down Parsing:

- Starts from the start symbol.
- May perform backtracking.
- Can be inefficient for ambiguous input.

Earley Parsing:

- Handles ambiguous input.
- Handles incomplete input.
- Maintains partial parse states.
- Suitable for interactive applications.

```
""")
```

```
print("Result:")
```

```
print("Earley parsing is more suitable for interactive NLP.")
```

OUTPUT:

TOP-DOWN AND EARLEY PARSING

Incomplete Query:
best hotels near

Tokens:
['best', 'hotels', 'near']

EARLEY PARSER

Number of parses: 1

Partial/Available Parse:
(S (NP (ADJ best) (NOUN hotels) (PP (PREP near))))

COMPARISON

Top-Down Parsing:

- Starts from the start symbol.
- May perform backtracking.
- Can be inefficient for ambiguous input.

Earley Parsing:

- Handles ambiguous input.
- Handles incomplete input.
- Maintains partial parse states.
- Suitable for interactive applications.

Result:
Earley parsing is more suitable for interactive NLP.

> |

Q3: CFG, PCFG AND NEURAL PARSER COMPARISON

```
print("=" * 60)
print("CFG VS PCFG VS NEURAL PARSER")
print("=" * 60)
```

sentence = "I saw the man with a telescope."

```
print("\nAmbiguous Sentence:")
print(sentence)
```

```
# -----
# CFG
# -----
```

```
print("\n" + "=" * 60)
print("1. CFG")
print("=" * 60)
```

```
print("""
CFG can generate multiple valid structures.
```

Interpretation 1:
I saw [the man with a telescope]

Interpretation 2:

I saw [the man] [with a telescope]
""")

```
print("CFG does not assign probabilities.")
```

```
# -----  
# PCFG  
# -----
```

```
print("\n" + "=" * 60)  
print("2. PCFG")  
print("=" * 60)
```

```
grammar_rules = {  
    "Parse 1": 0.70,  
    "Parse 2": 0.30  
}
```

```
for parse, probability in grammar_rules.items():  
    print(parse, "->", probability)
```

```
best_parse = max(  
    grammar_rules,  
    key=grammar_rules.get  
)
```

```
print("\nMost probable parse:", best_parse)
```

```
# -----  
# Neural Parser  
# -----
```

```
print("\n" + "=" * 60)  
print("3. NEURAL CONSTITUENCY PARSER")  
print("=" * 60)
```

```
print("A neural parser learns patterns from training data.")  
print("It can handle complex language patterns.")  
print("It usually provides strong practical accuracy.")
```

```
# -----  
# Comparison  
# -----
```

```
print("\n" + "=" * 60)  
print("COMPARISON")  
print("=" * 60)
```

```
print("""  
CFG:  
Interpretability = High  
Ambiguity Ranking = No
```

```
PCFG:  
Interpretability = High
```

Ambiguity Ranking = Yes

Neural Parser:

Interpretability = Lower

Practical Accuracy = High

""")

```
print("Result:")
```

```
print("PCFG provides a good balance between")
```

```
print("interpretability and practical accuracy.")
```

OUTPUT:

```
=====
CFG VS PCFG VS NEURAL PARSER
=====
```

Ambiguous Sentence:

I saw the man with a telescope.

```
=====
1. CFG
=====
```

CFG can generate multiple valid structures.

Interpretation 1:

I saw [the man with a telescope]

Interpretation 2:

I saw [the man] [with a telescope]

CFG does not assign probabilities.

```
=====
2. PCFG
=====
```

Parse 1 -> 0.7

Parse 2 -> 0.3

Most probable parse: Parse 1

3. NEURAL CONSTITUENCY PARSER

A neural parser learns patterns from training data.
It can handle complex language patterns.
It usually provides strong practical accuracy.

COMPARISON

CFG:

Interpretability = High
Ambiguity Ranking = No

PCFG:

Interpretability = High
Ambiguity Ranking = Yes

Neural Parser:

Interpretability = Lower
Practical Accuracy = High

Result:

PCFG provides a good balance between
interpretability and practical accuracy.

>|

Q4: FEATURE STRUCTURES IN MULTILINGUAL NLP

```
print("=" * 60)
print("FEATURE STRUCTURE ANALYSIS")
print("=" * 60)
```

```
# -----
# Word feature structures
# -----
```

```
noun = {
    "word": "boy",
    "category": "Noun",
    "number": "Singular",
    "person": "Third",
    "gender": "Male"
}
```

```
verb = {
    "word": "plays",
    "category": "Verb",
    "number": "Singular",
    "person": "Third",
    "tense": "Present"
}
```

```
print("\nNOUN FEATURES")
print("-" * 60)
```

```
for key, value in noun.items():
    print(key, ":", value)
```

```

print("\nVERB FEATURES")
print("-" * 60)

for key, value in verb.items():
    print(key, ":", value)

# -----
# Agreement checking
# -----

print("\n" + "=" * 60)
print("FEATURE AGREEMENT")
print("=" * 60)

if noun["number"] == verb["number"]:

    print("Number Agreement: CORRECT")

else:

    print("Number Agreement: ERROR")

if noun["person"] == verb["person"]:

    print("Person Agreement: CORRECT")

else:

    print("Person Agreement: ERROR")

# -----
# Incorrect example
# -----

wrong_verb = {
    "word": "play",
    "number": "Plural",
    "person": "Third"
}

print("\n" + "=" * 60)
print("INCORRECT EXAMPLE")
print("=" * 60)

print("boy + play")

if noun["number"] != wrong_verb["number"]:
    print("Number Agreement Error")

# -----
# Comparison
# -----

```

```
print("\n" + "=" * 60)
print("CFG VS FEATURE STRUCTURES")
print("=" * 60)
```

```
print("""
CFG:
- Represents basic grammatical structure.
```

```
Feature Structures:
- Represent grammatical structure.
- Store number.
- Store gender.
- Store person.
- Store tense.
- Enforce grammatical agreement.
""")
```

```
print("Result:")
print("Feature structures are useful for multilingual NLP")
print("because they can represent grammatical constraints.")
```

OUTPUT:

=====

FEATURE STRUCTURE ANALYSIS

=====

NOUN FEATURES

word : boy
category : Noun
number : Singular
person : Third
gender : Male

VERB FEATURES

word : plays
category : Verb
number : Singular
person : Third
tense : Present

=====

FEATURE AGREEMENT

=====

Number Agreement: CORRECT
Person Agreement: CORRECT

=====

INCORRECT EXAMPLE

=====

boy + play
Number Agreement Error

=====

CFG VS FEATURE STRUCTURES

=====

CFG:

- Represents basic grammatical structure.

Feature Structures:

- Represent grammatical structure.
- Store number.
- Store gender.
- Store person.
- Store tense.
- Enforce grammatical agreement.

Result:

Feature structures are useful for multilingual NLP
because they can represent grammatical constraints.

Q5: SUBCATEGORIZATION FRAMES

```
print("=" * 60)
print("SUBCATEGORIZATION FRAME ANALYSIS")
print("=" * 60)
```

```
# -----
# Verb frames
# -----
```

```
frames = {

    "give": [
        "Subject",
        "Object",
        "Recipient"
    ],

    "sleep": [
        "Subject"
    ],

    "put": [
        "Subject",
        "Object",
        "Location"
    ]
}
```

```
# -----
# Display frames
# -----
```

```
for verb, arguments in frames.items():
```

```
    print("\nVerb:", verb)
```

```
    print("Required arguments:")
```

```
    for argument in arguments:
        print("  ->", argument)
```

```
# -----
# Example 1
# -----
```

```
print("\n" + "=" * 60)
print("EXAMPLE 1")
print("=" * 60)
```

```
print("Give the book to John.")
```

```
print("\nAction    : GIVE")
print("Object     : Book")
print("Recipient  : John")
```



```
# -----  
# Example 2  
# -----
```

```
print("\n" + "=" * 60)  
print("EXAMPLE 2")  
print("=" * 60)
```

```
print("John sleeps.")
```

```
print("\nAction   : SLEEP")  
print("Subject   : John")  
print("Object    : Not Required")
```

```
# -----  
# Example 3  
# -----
```

```
print("\n" + "=" * 60)  
print("EXAMPLE 3")  
print("=" * 60)
```

```
print("Put the file on the desk.")
```

```
print("\nAction   : PUT")  
print("Object    : File")  
print("Location   : Desk")
```

```
# -----  
# Invalid example  
# -----
```

```
print("\n" + "=" * 60)  
print("INVALID EXAMPLE")  
print("=" * 60)
```

```
print("Put the file.")
```

```
print("\nERROR:")  
print("The verb 'put' requires a location.")
```

```
# -----  
# Final result  
# -----
```

```
print("\n" + "=" * 60)  
print("RESULT")  
print("=" * 60)
```

```
print("Subcategorization frames identify the arguments")  
print("required by different verbs.")
```

```
print("\nThey help voice assistants understand:")
```

```
print("- Actions")  
print("- Objects")  
print("- Recipients")  
print("- Locations")
```

OUTPUT:

SUBCATEGORIZATION FRAME ANALYSIS

Verb: give

Required arguments:

- > Subject
- > Object
- > Recipient

Verb: sleep

Required arguments:

- > Subject

Verb: put

Required arguments:

- > Subject
- > Object
- > Location

EXAMPLE 1

Give the book to John.

Action : GIVE

Object : Book

Recipient : John

EXAMPLE 2

John sleeps.

Action : SLEEP

Subject : John

Object : Not Required

=====

EXAMPLE 3

=====

Put the file on the desk.

Action : PUT
Object : File
Location : Desk

=====

INVALID EXAMPLE

=====

Put the file.

ERROR:
The verb 'put' requires a location.

=====

RESULT

=====

Subcategorization frames identify the arguments required by different verbs.

They help voice assistants understand:

- Actions
- Objects
- Recipients
- Locations

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear	Shows reasonable interpretation	Basic interpretation; lacks depth	No meaningful interpretation

		interpretation and reasoning	with minor gaps		
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1	3	4	4	4	3	18	92
2	4	3	4	3	3	18	
3	3	4	3	4	3	20	
4	4	4	4	3	3	18	
5	3	3	4	4	4	18	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____